

Learning Report – Module 11: DevOps and CI/CD

Overview

This module introduced me to the fundamentals of DevOps and the importance of collaboration between Development and Operations teams. I learned how DevOps helps organizations deliver software faster, improve quality, and automate the software development lifecycle. The module also covered the concepts of Continuous Integration (CI) and Continuous Delivery/Deployment (CD), along with an overview of popular CI/CD tools used in the industry.

Learning Outcomes

After completing this module, I was able to:

- Understand the concept and importance of DevOps.
- Explain the goals and benefits of DevOps.
- Understand Continuous Integration (CI) and Continuous Delivery/Deployment (CD).
- Differentiate between CI and CD.
- Identify popular DevOps and CI/CD tools.
- Understand how automation improves software development and deployment.

Introduction to DevOps

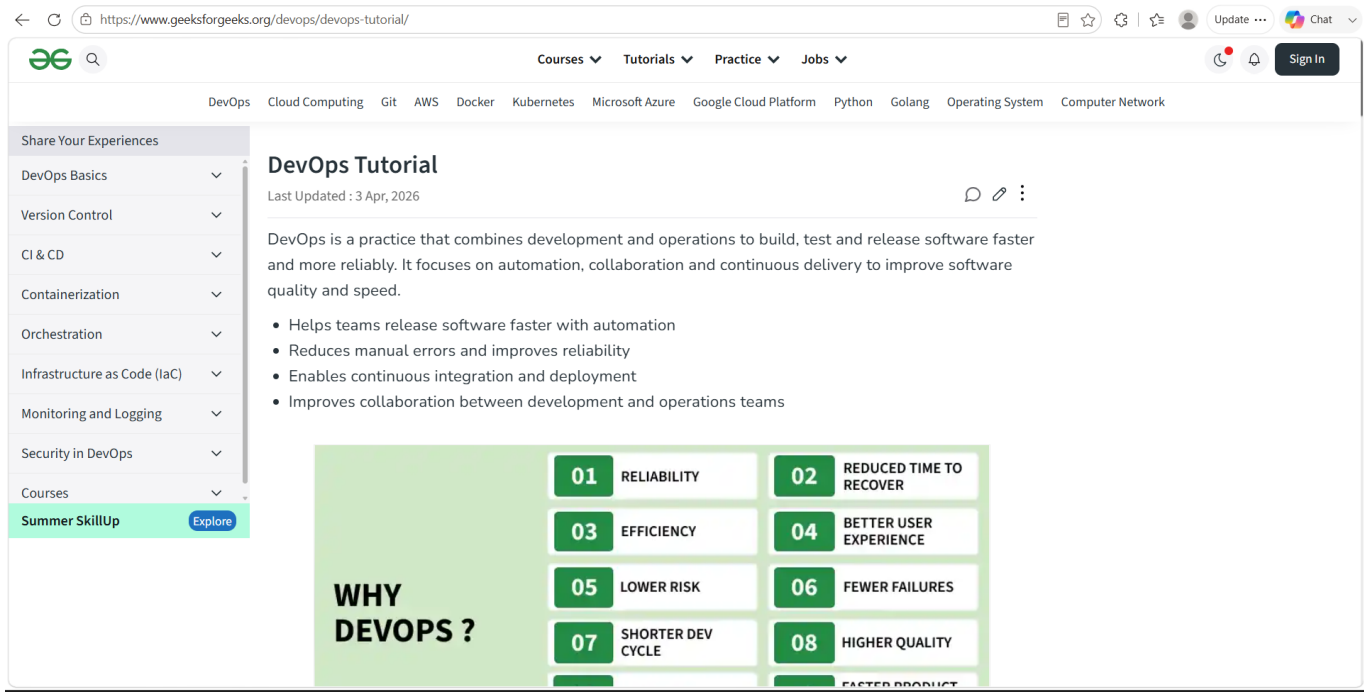
DevOps is a software development approach that combines Development (Dev) and Operations (Ops) to improve collaboration and automate the software delivery process. It helps organizations develop, test, and deploy applications more efficiently while maintaining high quality.

Goals of DevOps

- Faster software delivery.
- Improved collaboration between teams.
- Reduced deployment failures.
- Better software quality.
- Continuous monitoring and feedback.

Benefits of DevOps

- Faster release cycles.
- Increased productivity.
- Improved collaboration.
- Better application reliability.
- Reduced manual work through automation



Understanding CI/CD

Continuous Integration (CI)

Continuous Integration is the practice of regularly merging code changes into a shared repository. Every code change is automatically built and tested to detect errors early in the development process.

Continuous Delivery (CD)

Continuous Delivery ensures that the application is always ready for deployment. After successful testing, the software can be released to production with minimal manual effort.

Continuous Deployment

Continuous Deployment takes automation one step further by automatically deploying every successful code change to the production environment without manual approval.

Difference between CI and CD

Continuous Integration (CI)	Continuous Delivery/Deployment (CD)
Focuses on integrating code frequently.	Focuses on delivering or deploying the application.
Automatically builds and tests code.	Automatically prepares or deploys the application.
Detects bugs early.	Speeds up software releases.

← ↻ 🔍 <https://www.geeksforgeeks.org/devops/what-is-ci-cd/> 📄 ⭐ ⚙️ 🔔 🗨️ Update ... Chat

🌐 **GeeksforGeeks** Courses ▾ Tutorials ▾ Practice ▾ Jobs ▾

DevOps Cloud Computing Git AWS Docker Kubernetes Microsoft Azure Google Cloud Platform Python Golang Operating System Computer Network

Share Your Experiences

- DevOps Basics ▾
- Version Control ▾
- CI & CD ▾
- Containerization ▾
- Orchestration ▾
- Infrastructure as Code (IaC) ▾
- Monitoring and Logging ▾
- Security in DevOps ▾
- Courses ▾
- Summer SkillUp [Explore](#)

Introduction to CI/CD

Last Updated : 18 Jun, 2026

CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a modern software development practice that automates the process of building, testing, and releasing applications. It plays a key role in DevOps by streamlining collaboration between development and operations teams.

- Automates code integration, testing, and deployment workflows.
- Reduces manual effort while improving software quality.
- Smaller, frequent updates replace large risky releases.

CI/CD Before Adoption

Before the adoption of CI/CD, software development and delivery were slow, manual, and prone to errors. Processes were less automated, making releases time-consuming and risky.

```

graph LR
    A[WRITE CODE  
(Separate Branches)] --> B[MANUAL BUILD]
  
```

BEFORE CI/CD

Benefits of CI/CD

- Faster software development.
- Early bug detection.
- Automated testing and deployment.
- Reduced manual effort.
- Faster and more reliable releases.

Popular CI/CD Tools

During this module, I learned about commonly used DevOps tools, including:

- **Jenkins** – An open-source automation server used for building, testing, and deploying applications.

- **GitHub Actions** – A CI/CD platform integrated with GitHub that automates workflows directly from repositories.
- **GitLab CI/CD** – Built-in CI/CD features provided by GitLab for automated software delivery.
- **CircleCI** – A cloud-based CI/CD platform that automates software development workflows.

These tools help automate repetitive tasks such as building, testing, and deploying applications, making software delivery faster and more reliable.

The screenshot shows a web browser at the URL <https://www.geeksforgeeks.org/devops/introduction-to-devops/>. The page features a navigation bar with various technology topics like DevOps, Cloud Computing, Git, AWS, Docker, Kubernetes, Microsoft Azure, Google Cloud Platform, Python, Golang, Operating System, and Computer Network. A sidebar on the left lists categories such as DevOps Basics, Version Control, CI & CD, Containerization, and more. The main content area includes a diagram of the DevOps lifecycle, which is a circular flow involving the Development Team and the Operation Team. The stages are: CODE, PLAN, RELEASE, DEPLOY, OPERATE, MONITOR, TEST, and BUILD. Below the diagram, the text states 'Stages of DevOps are:' followed by the 'Plan Stage' description: 'Defining project goals, requirements, and task breakdowns to align development and operations teams.' and a bullet point: 'Work is broken down into tasks and user stories to ensure clarity and alignment.' On the right side of the page, there is a Microsoft Azure advertisement with the text 'Users spike. Stress doesn't. Keep your performance steady with Azure autoscaling.' and a 'Try Azure for free' button.

Conclusion

This module provided a clear understanding of DevOps and the role it plays in modern software development. I learned how Continuous Integration and Continuous Delivery/Deployment help automate the software development lifecycle, improve collaboration, and ensure faster, high-quality software releases. I also gained an overview of popular CI/CD tools such as Jenkins, GitHub Actions, GitLab CI/CD, and CircleCI. Overall, this module helped me understand how DevOps practices improve software development efficiency, reliability, and scalability.